Endless Runner — Task 4

Conversation transcript: sound effects, updated code, and WAV setup

Scope note: This PDF compiles the visible conversation in this chat about Task 4. It includes the user requests and the substantive assistant replies, including the full code provided in the chat. It does not include hidden system instructions or internal tool messages.

1. User request — Implement Task 4

Implement Task 4 of the Endless Runner project: add sound effects for jumping, scoring, and game over.

Requirements: play a jump sound when the player successfully jumps; play a scoring sound when the player passes an obstacle and earns a point; play a game-over sound on collision; use existing sound assets or add suitable short files; initialize Pygame mixer safely; avoid repeated game-over sounds; handle missing audio gracefully; preserve collision detection, speed cap, Game Over screen, final score, replay, difficulty selection, and Exit; list modified files and testing steps.

2. Assistant reply — Task 4 implementation overview

The assistant provided an updated project ZIP and explained the intended changes:

Files: main.py; game/game_engine.py; game/player.py; sounds/jump.wav; sounds/score.wav; sounds/game_over.wav. The ZIP also included game/obstacle.py and game/__init__.py.

Sound behavior: jump.wav plays only when a jump succeeds; score.wav plays once when an obstacle is passed and a point is awarded; game_over.wav plays on the collision transition, not every frame.

Suggested structure:

Project structure

```
your_project/
■■■ main.py
■■■ game/
■   ■■■ __init__.py
■   ■■■ game_engine.py
■   ■■■ player.py
■   ■■■ obstacle.py
■■■ sounds/
    ■■■ jump.wav
    ■■■ score.wav
    ■■■ game_over.wav
```

Testing: run python main.py; test a grounded jump, pass an obstacle, collide with an obstacle, replay using a difficulty, and select Exit. The assistant noted that syntax compilation succeeded but a full local gameplay test was not performed in its execution environment.

3. User request — Give all updated codes instead

The user asked for all updated code files directly in the chat rather than only a ZIP.

Assistant reply — main.py

main.py

```
import pygame
from game.game_engine import GameEngine

# Initialize Pygame.
# Audio initialization is handled safely inside GameEngine.
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 800, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Endless Runner - Pygame Version")

# Background color
SKY = (200, 220, 240)

# Clock
clock = pygame.time.Clock()
FPS = 60


def main():
    engine = GameEngine(WIDTH, HEIGHT, difficulty="medium")
    running = True

    while running:
        SCREEN.fill(SKY)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
                continue

            result = engine.handle_event(event)

            if result == "exit":
                running = False
                break

            if result in ("easy", "medium", "hard"):
                # Create a fresh game with the selected difficulty.
                engine = GameEngine(
                    WIDTH,
                    HEIGHT,
                    difficulty=result
                )

        if not running:
            break

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

game/game_engine.py

```
from pathlib import Path

import pygame

from .player import Player
from .obstacle import Obstacle
```

```
# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)
YELLOW = (255, 220, 0)
RED = (210, 45, 45)


# Difficulty settings
DIFFICULTIES = {
    "easy": {
        "start_speed": 4.0,
        "max_speed": 9.0,
        "speed_increase": 0.002,
        "spawn_interval": 90,
    },
    "medium": {
        "start_speed": 6.0,
        "max_speed": 12.0,
        "speed_increase": 0.003,
        "spawn_interval": 70,
    },
    "hard": {
        "start_speed": 8.0,
        "max_speed": 16.0,
        "speed_increase": 0.004,
        "spawn_interval": 50,
    },
}


class GameEngine:
    def __init__(self, width, height, difficulty="medium"):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        difficulty = difficulty.lower()
        if difficulty not in DIFFICULTIES:
            difficulty = "medium"

        self.difficulty = difficulty

        # Load sounds safely. The game still works without audio.
        self.sounds = self._load_sounds()

        settings = DIFFICULTIES[self.difficulty]

        self.start_speed = settings["start_speed"]
        self.max_speed = settings["max_speed"]
        self.speed_increase_per_frame = settings["speed_increase"]
        self.spawn_interval = settings["spawn_interval"]

        # Player
        self.player = Player(80, self.ground_y)

        # Game state
        self.speed = self.start_speed
        self._spawn_timer = 0
        self.obstacles = []
        self.distance = 0
        self.score = 0
        self.game_over = False
        self._game_over_logged = False

        # Fonts
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over_font = pygame.font.SysFont(
            "Arial", 64, bold=True
        )
        self.final_score_font = pygame.font.SysFont(
            "Arial", 36, bold=True
        )
        self.menu_font = pygame.font.SysFont(
            "Arial", 30, bold=True
        )
        self.help_font = pygame.font.SysFont("Arial", 18)

        # Game-over menu
        self.menu_options = ["Easy", "Medium", "Hard", "Exit"]
```

```
        self.selected_option = 0

    @staticmethod
    def _load_sounds():
        """Initialize the mixer and load available sound files."""
        sounds = {
            "jump": None,
            "score": None,
            "game_over": None,
        }

        try:
            # Initialize the mixer if it has not already been initialized.
            if not pygame.mixer.get_init():
                pygame.mixer.init()

            sounds_dir = (
                Path(__file__).resolve().parent.parent / "sounds"
            )

            filenames = {
                "jump": "jump.wav",
                "score": "score.wav",
                "game_over": "game_over.wav",
            }

            for name, filename in filenames.items():
                sound_path = sounds_dir / filename

                try:
                    if sound_path.is_file():
                        sounds[name] = pygame.mixer.Sound(
                            str(sound_path)
                        )
                    else:
                        print(
                            f"Sound file not found: {sound_path}"
                        )

                except (pygame.error, OSError) as exc:
                    print(
                        f"Could not load {filename}: {exc}"
                    )

        except (pygame.error, OSError) as exc:
            print(
                f"Audio unavailable. Running without sound: {exc}"
            )

        return sounds

    def _play_sound(self, name):
        """Play a sound if it loaded successfully."""
        sound = self.sounds.get(name)

        if sound is None:
            return

        try:
            sound.play()
        except pygame.error:
            # Continue running if audio becomes unavailable.
            pass

    def handle_event(self, event):
        if event.type != pygame.KEYDOWN:
            return None

        # Handle keyboard input on the Game Over screen.
        if self.game_over:
            if event.key in (pygame.K_UP, pygame.K_w):
                self.selected_option = (
                    self.selected_option - 1
                ) % len(self.menu_options)

            elif event.key in (pygame.K_DOWN, pygame.K_s):
                self.selected_option = (
                    self.selected_option + 1
                ) % len(self.menu_options)

            elif event.key in (
```

```
                pygame.K_RETURN,
                pygame.K_KP_ENTER,
                pygame.K_SPACE,
            ):
                choice = self.menu_options[
                    self.selected_option
                ].lower()

                if choice == "exit":
                    return "exit"

                return choice

            return None

        # Play the jump sound only when a jump succeeds.
        if event.key in (
            pygame.K_SPACE,
            pygame.K_UP,
            pygame.K_w,
        ):
            if self.player.jump():
                self._play_sound("jump")

        return None

    def handle_input(self):
        # Jump input is handled by handle_event().
        pass

    def update(self):
        # Prevent repeated processing after game over.
        if self.game_over:
            return

        # Increase speed without exceeding the selected difficulty cap.
        self.speed = min(
            self.speed + self.speed_increase_per_frame,
            self.max_speed,
        )

        self.player.update()

        # Spawn obstacles.
        self._spawn_timer += 1

        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0

            self.obstacles.append(
                Obstacle(
                    self.width,
                    self.ground_y,
                    self.speed,
                )
            )

        # Move obstacles.
        for obstacle in self.obstacles:
            obstacle.move()
            obstacle.speed = self.speed

        # Collision detection uses the obstacle's full movement path.
        player_rect = self.player.rect()

        for obstacle in self.obstacles:
            if obstacle.swept_rect().colliderect(player_rect):
                # This transition happens once. Subsequent updates return
                # immediately because game_over is now True.
                self.game_over = True
                self._play_sound("game_over")
                return

        # Award points and play the scoring sound exactly once per obstacle.
        for obstacle in self.obstacles:
            if (
                not obstacle.scored
                and obstacle.x + obstacle.width < self.player.x
            ):
                obstacle.scored = True
                self.score += 1
```

```
                self._play_sound("score")

        # Remove obstacles that have left the screen.
        self.obstacles = [
            obstacle
            for obstacle in self.obstacles
            if not obstacle.off_screen()
        ]

        self.distance += self.speed

    def render(self, screen):
        # Draw ground and player.
        pygame.draw.line(
            screen,
            BROWN,
            (0, self.ground_y),
            (self.width, self.ground_y),
            4,
        )

        pygame.draw.rect(
            screen,
            WHITE,
            self.player.rect(),
        )

        # Draw obstacles.
        for obstacle in self.obstacles:
            pygame.draw.rect(
                screen,
                DARK_GREEN,
                obstacle.rect(),
            )

        # Draw current score.
        score_text = self.font.render(
            f"Score: {self.score}",
            True,
            BLACK,
        )
        screen.blit(score_text, (10, 10))

        if self.game_over:
            # Dark overlay.
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA,
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            # Game-over title.
            title = self.game_over_font.render(
                "GAME OVER",
                True,
                RED,
            )
            screen.blit(
                title,
                title.get_rect(
                    center=(self.width // 2, 95)
                ),
            )

            # Final score.
            final_score = self.final_score_font.render(
                f"Final Score: {self.score}",
                True,
                WHITE,
            )
            screen.blit(
                final_score,
                final_score.get_rect(
                    center=(self.width // 2, 155)
                ),
            )

            # Difficulty selection and Exit menu.
            for index, option in enumerate(self.menu_options):
                selected = index == self.selected_option
```

```
                color = YELLOW if selected else WHITE

                label = (
                    f"> {option} <"
                    if selected
                    else option
                )

                option_text = self.menu_font.render(
                    label,
                    True,
                    color,
                )

                screen.blit(
                    option_text,
                    option_text.get_rect(
                        center=(
                            self.width // 2,
                            210 + index * 38,
                        )
                    ),
                )

            # Keyboard instructions.
            help_text = self.help_font.render(
                "UP/DOWN or W/S to choose, ENTER to confirm",
                True,
                WHITE,
            )
            screen.blit(
                help_text,
                help_text.get_rect(
                    center=(
                        self.width // 2,
                        self.height - 18,
                    )
                ),
            )

            # Print final score once, not every frame.
            if not self._game_over_logged:
                print(
                    "Game over! Final score:",
                    self.score,
                )
                self._game_over_logged = True
```

game/player.py

```
import pygame


class Player:
    def __init__(self, x, ground_y, width=30, height=40):
        self.x = x
        self.ground_y = ground_y
        self.width = width
        self.height = height

        self.y = ground_y - height
        self.vy = 0

        self.gravity = 0.8
        self.jump_strength = -15
        self.on_ground = True

    def jump(self):
        """Start a jump only when grounded."""
        if not self.on_ground:
            return False

        self.vy = self.jump_strength
        self.on_ground = False
        return True

    def update(self):
        self.vy += self.gravity
        self.y += self.vy
```

```
        ground_level = self.ground_y - self.height

        if self.y >= ground_level:
            self.y = ground_level
            self.vy = 0
            self.on_ground = True

    def rect(self):
        return pygame.Rect(
            self.x,
            self.y,
            self.width,
            self.height,
        )
```

game/obstacle.py

```
import pygame


class Obstacle:
    def __init__(
        self,
        x,
        ground_y,
        speed,
        width=25,
        height=40,
    ):
        self.x = x
        self.previous_x = x

        self.width = width
        self.height = height
        self.y = ground_y - height

        self.speed = speed
        self.scored = False

    def move(self):
        # Save the position before movement for collision detection.
        self.previous_x = self.x
        self.x -= self.speed

    def off_screen(self):
        return self.x + self.width < 0

    def rect(self):
        return pygame.Rect(
            self.x,
            self.y,
            self.width,
            self.height,
        )

    def swept_rect(self):
        """Return a rectangle covering the obstacle's movement this frame."""
        left = min(self.previous_x, self.x)
        right = max(self.previous_x, self.x) + self.width

        return pygame.Rect(
            left,
            self.y,
            right - left,
            self.height,
        )
```

4. User question — What needs to be in the WAV files?

The user asked what the .wav files need to contain.

Assistant reply — WAV contents

The WAV files should contain actual short audio recordings, not text or Python code:

jump.wav: a short, light upward swoosh or cartoon jump sound; suggested duration 0.1–0.4 seconds.

score.wav: a quick positive beep, ding, or coin-collection sound; suggested duration 0.1–0.5 seconds.

game_over.wav: a descending tone, failure buzzer, or short losing sound; suggested duration 0.5–1.5 seconds.

Suggested sources were Pixabay Sound Effects (https://pixabay.com/sound-effects/) and Freesound (https://freesound.org/), with a reminder to check each sound's licence.

Suggested search terms: “short game jump sound effect”, “8-bit coin ding sound effect”, and “retro game over sound effect”.

The response recommended converting downloaded files to WAV if necessary, naming them exactly jump.wav, score.wav, and game_over.wav, and putting them in the root-level sounds folder. Standard PCM WAV files were recommended for compatibility.

5. Quick testing checklist

• Run python main.py from the project root.

• Press Space while grounded: jump.wav should play. Press again mid-air: no new jump should begin or play.

• Pass an obstacle: score increases by one and score.wav plays once.

• Collide with an obstacle: game_over.wav plays once and the final score/menu appear.

• Choose Easy, Medium, or Hard and press Enter: a fresh game starts.

• Choose Exit and press Enter: the application closes.

• If audio files or the audio device are unavailable, the game should still run without sound.